

プログラミング演習 3

最終課題レポート

学部学科

情報工学部情報工学科

提出日

2023 年 8 月 9 日

1. CLI アプリケーションによるバイナリエディタ

1.1 実装方法・内容

CLI アプリケーションとして実装するにあたり、処理構造は大きく三層に分けられる。はじめに、CLI 上で図 1.0 のコマンドを実行する。ここでは、ユーザーがプロジェクト内の bin フォルダでコマンドを実行することを想定している。

続いて、引数として受け渡されたバイナリファイルの内容に対して処理を行う。ファイルを開き、ファイルインプットストリームとバッファインプットストリームでパイプを行い、読み取りを行う。このとき、ユーザーによる書き込みは想定していない。read 関数による読み込みを 16 回行ったタイミングで一行ずつ出力を行う。出力のフォーマットは、一列目の出力回数を示す八桁番号を主キーとして行を特定できるようにしてある。また、二列目と三列目にそれぞれ 16 進数・ASCII の順で表示を行う。

最後に、256 個分の 16 進数 2 ビットを ASCII に変換する間隔で、ユーザーから Enter キー押下による続行を求める。これを繰り返すことで、バイナリファイルの読み取り・変換・出力を行う。

```
C:\Users\Owner\bin> java Task8_CLI [バイナリファイルへのパス]
```

図 1.0 CLI 上で実行したコマンド

2. GUI アプリケーションによるバイナリエディタ

2.1 実装方法・内容

図 2.1 のようなウィンドウを用意した。ウィンドウ上部にあるメニューから File を選択すると、Close と Choose の二つを選ぶことが可能であり、ファイル選択を行うには Choose を選択する。エクスプローラーから任意のバイナリファイルを選択して、Go!! ボタンを押下することでプログラムが実行される。

動作の大枠は CLI と同様であるが、表示方法に差異がある。GUI では、256 文字を一気に表示させるのではなく、一行ずつ表示させるような出力となるようにした。また、スレッドによりファイル選択・出力の二者は平行で行うこともできる。また、出力した文章の行数が枠を超えてしまう場合には、サイドバーが出現する GUI コンポーネントを利用している。

また、追加課題としてプログレスバーの配置を行った。これによって、ユーザーは残りの処理にどれほど時間がかかるのかが直感的に分かる。

2.2 課題

本プログラムでは、いくつかの未解決の問題が残っている。一点目として、スレッドを利用して表示を行う際に正しく処理されないことがあることが挙げられる。原因は解明できていないが、内部の配列に対して例外処理がスローされていることから、変換処理の際に原因が発生していると考えられる。

二点目として、表示の速度に限界があることが挙げられる。内部では、一行の出力を行うごとに 50ms のスリープメソッドをスレッドに対して行うことで、20(行/s)の速度で表示するようになっている。しかし、これ以上の速度で出力を行おうとするとエラーが発生する。この原因として考えられるのが、関数間における処理能力の違いである。確証は得られていないが、GUI コンポーネント TextArea に対する appendText メソッドがボトルネックになっているのではないかと考えられる。当該メソッドは変換処理スレッド内で多く用いられている。さらに CLI アプリケーションとの処理工程の差異から、以上のような考察となった。解決策として、appendText を用いないことが挙げられる。

三点目として、プログレスバーの有意性が現時点では全くないという点が挙げられる。配置したプログレスバーは、処理変換スレッドで文字列の出力が始まると動き出す仕様となっている。しかし実際は、1 秒ごとにゲージが変化、適当な位置で停止、処理完了時にゲージが満たされるという振る舞いをしている。解決策として、全体データに対する未処理データの割合でゲージを変化させる方法が考えられるが、任意のファイルに対して未処理データ・全体データを正確に把握する手段を実現することはできなかった。

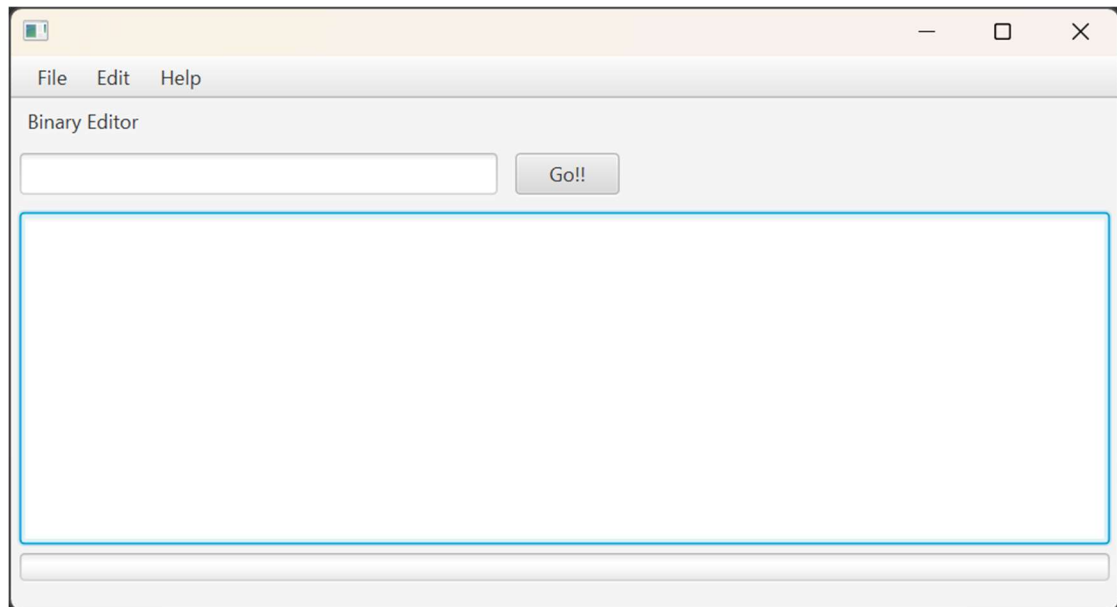


図 2.1 GUI によるバイナリ ASCII 変換ウィンドウ

3. モバイルアプリケーションによるバイナリエディタ

3.1 実装方法・内容

Android のモバイルアプリケーションとして実装するにあたり，図 3.1 のような GUI ウィンドウを用意した．FILE ボタンを押下すると，AVD 内のエクスプローラーによるメニュー操作でファイルの選択を行うことができる．また，ファイルを選択すると瞬時に出力を行い，選択したファイルの名前を FILE ボタンの横に表示させ，どのファイルに対して操作を行っているかが分かりやすくした．

内部では，取り出した 16 進数と変換後の ASCII 文字をそれぞれ連結させる変換・連結処理，変換・連結処理後に一行ずつ出力を行う処理の二段階に分かれている．また，縦に文章を拡張できる TextView である textMultiline と，HorizontalScrollView を組み合わせ，上下と左右を独立にスワイプすることを可能とした．

3.2 課題

本プログラムでは，いくつかの未解決の問題が残っている．一点目として，縦のスクロールがスムーズに行えない点が挙げられる．activity_main.xml では，横スクロールコンポーネントに縦スクロールコンポーネントを入れ子にした．その内側へさらに文章を挿入する

ビューを入れ子にすることで、上下左右を自由にスライドできる UI を作成しようとしたが、実際には実現できなかった。原因は特定できていないため推論となるが、縦スクロールを実現するコンポーネントである ScrollView と複数行に跨る TextView の textMultiLine の相性が悪かったため起こった不具合であると考えた。

二点目として、ファイルの格納先が不明であるという点である。試験に用いた png ファイルは、AVD のデバイスファイルエクスプローラー内に格納されており、図 3.2 に示したパスがその格納先を示している。しかし、実際にはファイルアプリ内でファイルを発見することができなかった。その解決策として、本プログラムでは PC から AVD の画面に移動させたい任意のファイルをドラッグアンドドロップすると、AVD のファイルアプリ内部にある Downloads フォルダに格納される機能を利用して行った。

三点目として、FILE ボタンと GO ボタンを独立させていない点が挙げられる。当初は、FILE ボタンからユーザーが任意のファイルを選択、GO ボタンから処理を実行する構成を目指したが、ファイルを選択するためのメニュー操作を行う機能から、文字列を取り出す処理を独立させることができず、本来二つに分けられるはずの処理が一体化してしまったことが原因である。

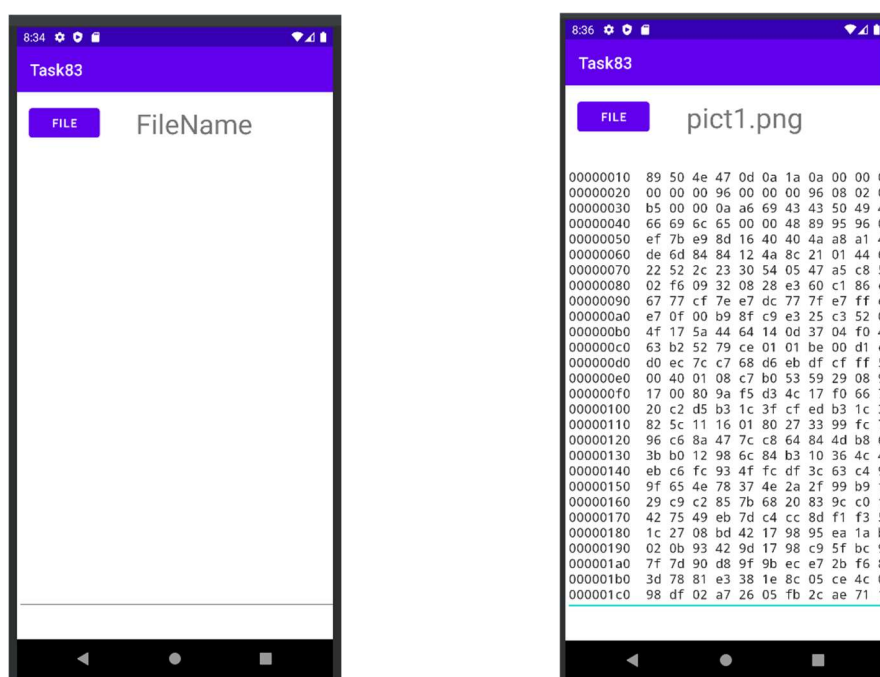


図 3.1 Android モバイルアプリによるバイナリ ASCII 変換ウィンドウ
(左図)処理前のスクリーン (右図)処理後のスクリーン

/data/data/jp.ac.meijo_u.id221205015.task83/files/pict1.png

図 3.2 AVD 内の試験用 png ファイル格納先

4. OpenCV を用いたモザイク加工処理の実装

4.1 実装について

本プログラムは完成しておらず、モザイク処理を実装することはできなかった。本来の趣旨とは大きく異なるが、なぜ正常に動作しないかを考察する。

4.2 準備

OpenCV とは、オープンソースの外部ライブラリで、画像や動画の分析を行うことができる。使用できる言語は、MATLAB, Python, Java, javascript の 4 種であり、今回は Java を使用して OpenCV ライブラリを利用した。

特に、Java は OpenCV を利用するために、JavaCV と呼ばれる別のライブラリをダウンロードする。ここから、各自の OS に合わせた jar ファイルを選ぶ。この際、OS 名がファイル名に入っている jar ファイル、platform という単語が入っている jar ファイルと、無印の jar ファイルの 3 つを用意しなくてはならない。用意した jar ファイルを、使用している IDE やエディタに合わせた方法で依存関係を明らかにしておく。今回、IDE として eclipse を使用した。これを行わないと、ロードエラーが発生してしまい、一切実行することができない。

また、OpenCV と JavaCV は、使用するライブラリで整合性を得るためにバージョンを揃える必要がある。今回は両者のバージョンを 4.7.0 に揃えた。これを行わないと、思わぬ処理結果を得ることになると考えられる。

最後に、OpenCV でダウンロードしたファイル内なる dll ファイルをシステム環境変数にパスを通すことで、使用しているエディタで使うことができるようになる。これを行わないと、jar ファイルと同様にロードエラーが発生してしまうため、必ず行う必要がある。

4.3 実装できた内容

javacv を用いて、ウェブカメラのキャプチャには成功した。そして、インターネットの情報を参照しながら顔のキャプチャに成功した。また、キャプチャするためのカスケード xml ファイルには haarcascade_frontalface_default.xml を使用している。また、このカスケード xml ファイルを任意の検出対象ごとに新たにダウンロードと配置を行うことで、検出対象の切り替えを行うことができる。ファイルの切り替え機能は実装できていない。

4.4 実装できていない内容

JavaFX を利用した、コンポーネント上での動画の再生が未実装である。これを実装させるには、Mat 型をコンポーネントで利用できるような何かしらの型に変換する必要がある。

また、モザイク処理を実装するために、50x50 pixel の範囲で左下隅の RGB 値を複製させる機能を実装しようとしていたが、未実装となっている。これを実装するためには、UByteIndexer についてバグ(IndexOutOfBoundsException)が起これないように調整し、put メソッド側で発生する配列の範囲を大幅に超えたアドレスへのアクセスが起これていると考えられるため、これを解決することが必要である。また、get メソッドでは、RGB 値を正常に取得できていると考えられるが、エラーが起こる直前に 0 の値を取得していることが共通項として挙げられる。したがって、エラーの原因の一因として、get メソッドも関係していると考えられるため、get メソッドについても調整を行う必要がある。

4.5 モザイク処理が実装できない理由の考察

get メソッドと put メソッドの引数として、指定しているパラメーターが適切でない可能性がある。その根拠として、put メソッドにおける極度に配列範囲を超えたアドレスへのアクセスがあったことや、get メソッドが 0 を取得したときにエラーが発生したことが挙げられる。

また、Rect 型の変数に対する Mat 型への変換や、その他の扱いに関して問題があると考えられる。その根拠として、Rect 型の forEach メソッド内で用いている変数 r を RGBConverter 関数で処理させるタイミングでエラーが起これていることが挙げられる。

5 期末課題に取り組んだ感想

期末試験で取り組んだアプリケーションの実装について、今までの総復習のように取り組むことができ非常に良かったと思っている。しかし、各課題において実装できなかった機能が多々あり、非常に悔しい思いもしている。

CLI では、AnyPressKey 関数で、任意のキーが押されたときに処理が進行するプログラムを組もうと考えていた。しかし、自力で実装することは叶わず、結局 Scanner を活用して Enter が押されただけ先に進める仕様に変更した。これを実装するために、ターミナルのコマンドを用いて AnyPressKey 関数を実現する方法や、JavaFX の(ActionEvent)型を用いて実装を検討したが、どれもうまくいかなかった。

GUI においても、ファイルブラウザをメニュー操作から行おうとした結果、File 型や Uri 型の概念を自分で理解するのが困難だったため、インターネットから多くの情報を参照することとなった。また、スレッドを用いて並列処理を実装した結果、そもそも処理がうまくいかないことが増えてしまった。*Contoroller.java における処理は短くすべきであり、メインのスレッドに早く切り替えるべきであるという情報にしたがって、変換処理を別スレッドにしたことに起因するが、本末転倒となってしまった。また、プログレスバーを設置したが、実装の時間的なコストに対し、対して意味のないオブジェになってしまった。

AndroidStudio についても、メニュー操作によるファイルブラウザが非常に難解で、大部分をインターネットから参照してしまった。また、ファイルブラウザに重きを置きすぎたせいで、ユーザーが非常に使いづらい UI になってしまい、こちらも本末転倒となってしまった。さらに、スクロール機能を左右上下と斜めで実装しようと考えていたが、自分で設計するには時間的リソースが十分でなく、斜めにスクロールすることは叶わなかった。

応用課題については、OpenCV について事前知識が全くない状態で着手したこともあり、ほとんどの機能を実装することができなかった。また、コードを書ける状態にするまでに長い時間をかけてしまったため、基本課題のようにコードを書く時間を多くとることができなかった。さらに、バージョンによってメソッドの使い方が異なっていること、JavaCV がないと満足にコードを書けないこと、外部ライブラリと標準ライブラリの扱い方の違いなど、多くのことに惑わされた。

今回の期末課題では今までのプログラミング演習とは大きく異なり、楽しかったという気持ちより悔しかったという気持ちが大部分を占めている。しかし反対に、外部ライブラリを自分のデバイスで使うための手順を実際に学べたこと、今後の目標ができたこと、自分に足りないスキルをしっかりと確認することができたことなど、自分のためになることも大いにあった。今後のプログラミング学習では、今回の悔しいという気持ちを活かせるような学習をしていきたいと考えている。

参考文献

[1]

Java で OpenCV を使う

<https://qiita.com/ketaro-m/items/86564ecb5737099d21f5>

[2]

Java OpenCV ～学習準備のまとめ～

<https://zenryokuservice.com/wp/2020/05/27/java-opencv>

[3]

Java + OpenCV で Web カメラ映像のリアルタイム顔認識を試みる

<https://piglet-march.net/2019/10/22/java-opencv>

[4]

java.lang.UnsatisfiedLinkError: no jniopenblas_nolapack in java 11 #1435

<https://github.com/bytedeco/javacv/issues/1435>

[5]

UnsatisfiedLinkError Can not find jnivideoInputLib.dll #864

<https://github.com/bytedeco/javacv/issues/864>

[6]

opencv で扱える全てのクラスに関するドキュメント

<https://docs.opencv.org/3.4.16/javadoc/allclasses.html>

[7]

OpenCV2 系 (Java) でモザイク処理

<https://qiita.com/ciela/items/f29fb91e28360b514307>

[8]

opencv 利用のメモ

<http://labs.uchicom.com/javacv/>

[9]

Java で Mat の RGB 要素にアクセス・値を変更する方法

<https://teratail.com/questions/beftna75o28szb>

[10]

JavaCV で矩形に切り出し

<https://developers-trash.com/archives/298>

[11]

JavaCV(OpenCV)で画像の RGB を取得する

<https://developers-trash.com/archives/85>

[12]

ビデオキャプチャ オブジェクトの fps/フレーム/秒を取得する方法

<https://stackoverflow.com/questions/21853894/how-to-get-fps-frames-per-second-of-a-videocapture-object>

[13]

IndexOutOfBoundsException とは？

<https://programmer-life.work/java/indexoutofboundsexception>

[14]

OpenCV で始める簡単動画プログラミング第3版 著者 北山洋幸

出版 2018年6月20日 出版社 カットシステム